SELECT 5: Truy vấn con (Subqueries) trong MySQL

Khái niệm – Cú pháp – Giải thích – Ví dụ – Bài tập

DBMS Course

Smart Logistics and Supply Chain Management Lab

2026

Mục tiêu học tập

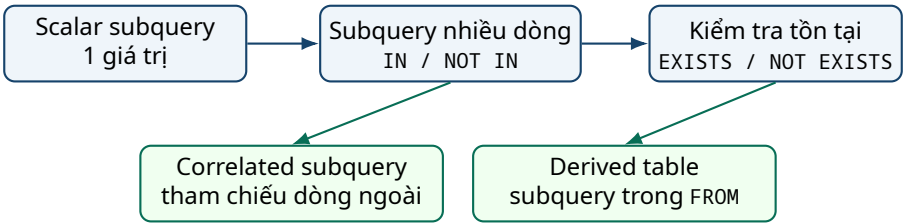
Nhận biết & giải thích

- Subquery là gì; xuất hiện ở những mệnh đề nào.
- Phân biệt kết quả trả về một giá trị và nhiều dòng.
- Hiểu truy vấn con **tương quan** và không tương quan.

Viết và kiểm tra SQL

- `IN`, `NOT IN`; `EXISTS`, `NOT EXISTS`.
- Subquery trong `SELECT`, `FROM`.
- Nhận diện lỗi `NULL`, sai số dòng và thiếu bí danh.

Mạch kiến thức của SELECT 5



Ngữ cảnh ví dụ: cơ sở dữ liệu `classicmodels`.

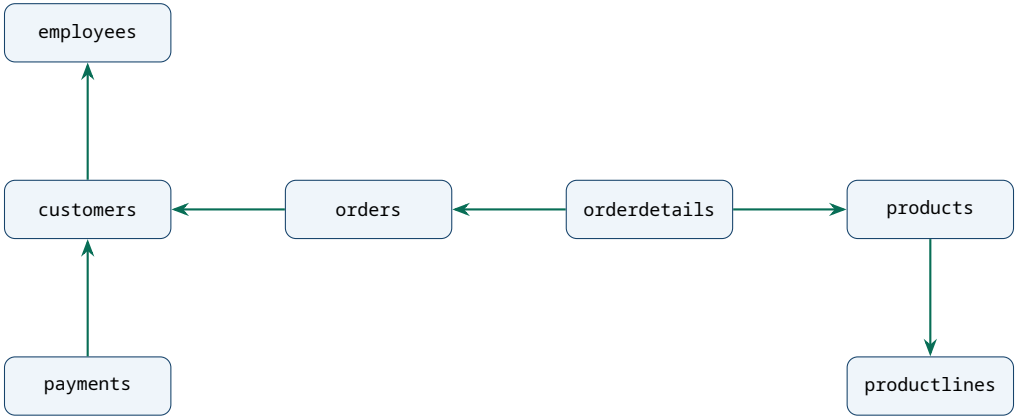
Các bảng cần dùng trong classicmodels

Bảng	Dữ liệu
customers	Khách hàng
orders	Đơn hàng
orderdetails	Chi tiết đơn
payments	Thanh toán

Bảng	Dữ liệu
products	Sản phẩm
productlines	Dòng sản phẩm
employees	Nhân viên
offices	Văn phòng

Khóa liên kết: `orders.customerNumber` → `customers.customerNumber`; `orderdetails.productCode` → `products.productCode`.

Quan hệ bảng phục vụ các truy vấn con



Mũi tên cho biết bảng chứa khóa ngoại → bảng được tham chiếu; điều kiện so sánh trong SQL phải sử dụng đúng cặp khóa.

Khởi động MySQL và khám phá bảng

```sql
USE classicmodels;
SHOW TABLES;
DESC products;
DESC customers;
DESC orders;
DESC orderdetails;
```

- `USE`: chọn CSDL đang làm việc.
- `SHOW TABLES`: liệt kê bảng.
- `DESC`: xem cột, kiểu dữ liệu và khóa.

Subquery là gì?

Định nghĩa

Truy vấn con (*subquery*) là một câu lệnh `SELECT` nằm trong câu truy vấn khác.

```sql
SELECT productCode, productName, buyPrice
FROM products
WHERE buyPrice > (
    SELECT AVG(buyPrice)
    FROM products
);
```

1. **Truy vấn trong:** tính giá mua trung bình.
2. **Truy vấn ngoài:** giữ những sản phẩm có giá lớn hơn ngưỡng đó.

Cách đọc truy vấn con theo hai tầng



Chú ý về thực thi

Đây là **cách hiểu logic**, không nhất thiết là kế hoạch thực thi vật lý: MySQL có thể tối ưu hóa hoặc viết lại truy vấn.

Subquery có thể xuất hiện ở đâu?

Vị trí	Vai trò	Ví dụ sử dụng
`WHERE`	Lọc dòng theo kết quả của câu khác	So sánh với AVG; IN / EXISTS
`SELECT`	Sinh thêm giá trị / cột	Đếm đơn hàng mỗi khách
`FROM`	Tạo derived table	Tổng doanh thu theo đơn
`HAVING`	Lọc nhóm bằng điều kiện con	So với trung bình các nhóm

Ghi nhớ

Kết quả của subquery có thể là **1 giá trị**, **nhiều dòng (1 cột)**, hoặc **một bảng nhiều cột**. Toán tử bên ngoài phải phù hợp với hình dạng kết quả.

Phân loại theo dạng kết quả và quan hệ

Tiêu chí	Các trường hợp
Số giá trị	Scalar (một giá trị); một cột nhiều dòng; bảng nhiều cột
Phụ thuộc	Non-correlated (độc lập); correlated (tham chiếu truy vấn ngoài)
Mục đích	So sánh; thành viên tập hợp; tồn tại; tạo cột; tạo bảng nguồn

Lưu ý

Không thể tùy ý dùng = với subquery có nhiều dòng. Trong MySQL, scalar subquery nhiều dòng sẽ phát sinh lỗi *Subquery returns more than 1 row*.

Scalar subquery: cú pháp và thành phần

```sql
SELECT column_list
FROM table_name
WHERE value_expression > (
    SELECT aggregate_function(column)
    FROM another_table
);
```

Giải thích cú pháp

- `(SELECT ...)`: câu truy vấn con trả về tối đa một dòng, một cột.
- `value_expression`: giá trị của dòng đang xét.
- `>`: toán tử so sánh, có thể thay bằng `=`, `<>`, `<`, `<=`, `>=`.

Ví dụ: giá mua cao hơn trung bình

```sql
SELECT productCode, productName, buyPrice
FROM products
WHERE buyPrice > (
    SELECT AVG(buyPrice)
    FROM products
);
```

Subquery

`AVG(buyPrice)` trả về một con số đại diện cho toàn bộ bảng.

Outer query

Kiểm tra `buyPrice > trung bình` trên từng sản phẩm.

Đầu ra: các sản phẩm đắt hơn mức giá mua trung bình (không gồm bằng ngưỡng).

Bước tính scalar subquery: ví dụ số

Dữ liệu giả lập để minh họa (không phải dữ liệu thật của classicmodels):

Sản phẩm	buyPrice	So với trung bình?
P1	30	Không
P2	50	Không
P3	70	Có
P4	90	Có

$$\text{AVG} = (30 + 50 + 70 + 90)/4 = 60$$

Ghi nhớ

Truy vấn ngoài chỉ giữ P3 và P4, vì 70 và 90 lớn hơn 60.

Ví dụ: tìm sản phẩm có MSRP cao nhất

```sql
SELECT productCode, productName, MSRP
FROM products
WHERE MSRP = (
    SELECT MAX(MSRP)
    FROM products
);
```

- Subquery trả **giá MSRP lớn nhất**.
- = giữ tất cả sản phẩm có đúng MSRP đó.
- Nếu nhiều sản phẩm cùng có MSRP tối đa, câu lệnh trả về **nhiều dòng** (hoàn toàn hợp lệ).

Ví dụ: khoản thanh toán lớn hơn trung bình

```sql
SELECT customerNumber, checkNumber,
       paymentDate, amount
FROM payments
WHERE amount > (
    SELECT AVG(amount)
    FROM payments
);
```

Ý nghĩa

Đưa ra các khoản thanh toán vượt mức trung bình. Khác với truy vấn trên `products`, nguồn dữ liệu được thay thành `payments`.

Bẫy: scalar subquery nhiều hơn một dòng

Viết sai:

```sql
SELECT customerNumber, customerName
FROM customers
WHERE customerNumber = (
    SELECT customerNumber
    FROM orders
);
```

Lưu ý

Truy vấn con có thể trả về nhiều mã khách hàng; trong MySQL, dùng = như vậy sẽ lỗi (trừ khi thực tế chỉ có tối đa 1 dòng).

Cách sửa: khi mục tiêu là thuộc tập giá trị, dùng `IN` hoặc `EXISTS`.

IN với subquery: cú pháp

```sql
SELECT column_list
FROM outer_table
WHERE key_column IN (
    SELECT matching_key
    FROM inner_table
);
```

Giải thích cú pháp

- Subquery phải trả về **một cột** để so khớp với một khóa.
- `IN` kiểm tra giá trị của dòng ngoài có thuộc tập con hay không.
- Các giá trị lặp trong tập con không làm dòng ngoài bị nhân bản.

IN: khách hàng đã từng đặt hàng

```sql
SELECT customerNumber, customerName
FROM customers
WHERE customerNumber IN (
    SELECT customerNumber
    FROM orders
);
```

- `orders` cung cấp tập mã khách hàng có đơn hàng.
- `customers` cung cấp thông tin tên khách hàng.
- Mỗi khách hàng được trả **tối đa một lần** nếu bảng customers có khóa chính duy nhất.

IN: sản phẩm đã từng được đặt

```sql
SELECT productCode, productName
FROM products
WHERE productCode IN (
    SELECT productCode
    FROM orderdetails
);
```

Ghi nhớ

Đây là truy vấn kiểu *membership*: một sản phẩm có thể nằm ở 100 dòng orderdetails nhưng vẫn chỉ xuất hiện một lần trong kết quả từ products.

IN và JOIN: có thể khác số dòng

IN

`products` là nguồn của kết quả. Mỗi sản phẩm phù hợp chỉ xuất hiện một lần (nếu khóa duy nhất).

INNER JOIN

Nếu nối trực tiếp `products` với `orderdetails`, một sản phẩm có thể lặp lại cho từng dòng chi tiết.

Lưu ý

Nếu mục tiêu là **tồn tại ít nhất một** đơn hàng, dùng `IN` hoặc `EXISTS` có thể biểu đạt ý định rõ hơn JOIN trực tiếp.

NOT IN: cú pháp và ý nghĩa

```sql
SELECT column_list
FROM outer_table
WHERE key_column NOT IN (
    SELECT matching_key
    FROM inner_table
);
```

Giải thích

- `NOT IN` chọn các giá trị **không thuộc** tập subquery.
- Hữu ích để tìm khách hàng chưa đặt, sản phẩm chưa được mua.
- Chỉ an toàn khi hiểu rõ khả năng xuất hiện `NULL` trong subquery.

NOT IN: sản phẩm chưa từng được đặt

```sql
SELECT productCode, productName
FROM products
WHERE productCode NOT IN (
    SELECT productCode
    FROM orderdetails
);
```

- Subquery: các mã sản phẩm đã được đặt.
- Truy vấn ngoài: sản phẩm không nằm trong tập này.
- Nếu `orderdetails.productCode` không `NULL`, cách viết này dễ hiểu.

Bẫy 3 giá trị: NOT IN và NULL

Giả sử truy vấn con trả về tập `(10, 20, NULL)`.

Điều kiện	SQL logic	Có vào kết quả?
`10 NOT IN (10,20,NULL)`	FALSE	Không
`30 NOT IN (10,20,NULL)`	UNKNOWN	Không
`30 NOT IN (10,20)`	TRUE	Có

Lưu ý

Mệnh đề `WHERE` chỉ giữ giá trị `TRUE`; `FALSE` và `UNKNOWN` đều bị loại. `NOT IN` với `NULL` có thể làm kết quả bất ngờ rỗng.

Cách sửa NOT IN: loại NULL ở truy vấn con

```sql
SELECT employeeNumber, lastName, firstName
FROM employees
WHERE employeeNumber NOT IN (
    SELECT salesRepEmployeeNumber
    FROM customers
    WHERE salesRepEmployeeNumber IS NOT NULL
);
```

Ghi nhớ

Bộ lọc `IS NOT NULL` ngăn giá trị NULL của nhân viên phụ trách làm sai nghĩa phép `NOT IN`.

Giải pháp thay thế: NOT EXISTS

```sql
SELECT e.employeeNumber, e.lastName, e.firstName
FROM employees AS e
WHERE NOT EXISTS (
    SELECT 1
    FROM customers AS c
    WHERE c.salesRepEmployeeNumber = e.employeeNumber
);
```

- Kiểm tra không có khách hàng nào liên kết với nhân viên đang xét.
- Không bị bẫy `NULL` ở bảng trong theo cách `NOT IN` gặp phải.

Subquery trong SELECT: cú pháp

```sql
SELECT o.key_column,
       o.descriptive_column,
       (
           SELECT COUNT(*)
           FROM inner_table AS i
           WHERE i.foreign_key = o.key_column
       ) AS relatedCount
FROM outer_table AS o;
```

Giải thích

- Mỗi dòng ngoài có thêm một **cột tính toán**.
- Subquery ở đây phải trả về **một giá trị** cho từng dòng ngoài.
- Đặt `AS relatedCount` để cột mới có tên dễ hiểu.

Ví dụ: số đơn hàng của mỗi khách hàng

```sql
SELECT c.customerNumber, c.customerName,
       (
           SELECT COUNT(*)
           FROM orders AS o
           WHERE o.customerNumber = c.customerNumber
       ) AS orderCount
FROM customers AS c;
```

- Vế `o.customerNumber = c.customerNumber` tạo sự liên kết với dòng ngoài.
- `COUNT(*)` bằng 0 nếu khách hàng không có đơn nào.
- Kết quả vẫn có **mọi khách hàng** trong bảng ngoài.

Ví dụ: tổng thanh toán của mỗi khách hàng

```sql
SELECT c.customerNumber, c.customerName,
       (
           SELECT SUM(p.amount)
           FROM payments AS p
           WHERE p.customerNumber = c.customerNumber
       ) AS totalPayment
FROM customers AS c;
```

Lưu ý

Với khách hàng không có khoản thanh toán, `SUM` trả `NULL`, không phải 0. Có thể dùng `COALESCE(..., 0)` nếu muốn hiển thị 0.

COALESCE: thay NULL bằng 0 khi tính tổng

```sql
SELECT c.customerNumber, c.customerName,
       COALESCE((
           SELECT SUM(p.amount)
           FROM payments AS p
           WHERE p.customerNumber = c.customerNumber
       ), 0) AS totalPayment
FROM customers AS c;
```

Ghi nhớ

`COALESCE(expression, 0)` lấy `expression` nếu khác NULL; ngược lại trả 0.

Subquery trong SELECT: những điều cần nhớ

- Subquery có thể sử dụng `COUNT`, `SUM`, `AVG` hoặc truy vấn một giá trị.
- Nếu truy vấn con trả nhiều dòng thì không dùng trực tiếp làm một ô dữ liệu.
- Subquery tương quan có thể đọc rõ hơn trong những bài toán *đếm / tính tổng theo từng đối tượng*.
- Với báo cáo dữ liệu lớn, có thể cân nhắc `JOIN + GROUP BY` hoặc bảng tổng hợp.

Correlated subquery là gì?

Định nghĩa

Correlated subquery tham chiếu cột hoặc bí danh của truy vấn ngoài, ví dụ `p.productLine` trong điều kiện của truy vấn con.

```sql
SELECT p.productCode, p.productName
FROM products AS p
WHERE p.buyPrice > (
    SELECT AVG(p2.buyPrice)
    FROM products AS p2
    WHERE p2.productLine = p.productLine
);
```

Ghi nhớ

Ngưỡng so sánh thay đổi theo **dòng sản phẩm** của mỗi `p`.

Non-correlated và correlated: so sánh

	Không tương quan	Tương quan
Tham chiếu	Chỉ cột của truy vấn con	Có cột của truy vấn ngoài
Ngưỡng	Có thể chung cho tất cả dòng	Có thể khác theo từng dòng
Ví dụ	So với AVG toàn bộ sản phẩm	So với AVG cùng productLine

Lưu ý

Đừng đồng nhất cách hiểu *"theo từng dòng* với việc DBMS chắc chắn chạy riêng subquery cho từng dòng. Bộ tối ưu MySQL có thể biến đổi cách thực thi.

Vì sao điều kiện tương quan rất quan trọng?

```sql
SELECT p.productCode, p.productName, p.productLine
FROM products AS p
WHERE p.MSRP > (
    SELECT AVG(p2.MSRP)
    FROM products AS p2
    WHERE p2.productLine = p.productLine
);
```

- Bỏ dòng `p2.productLine = p.productLine`: trở thành so với trung bình toàn bộ sản phẩm.
- Giữ dòng đó: so với trung bình **cùng dòng sản phẩm**.
- Hai câu hỏi kinh doanh khác nhau, có thể cho kết quả rất khác.

Minh họa nhóm: ngưỡng thay đổi theo từng dòng

Dữ liệu giả lập:

Sản phẩm	Line	Giá	Trung bình line	Đạt?
P1	Classic	100	75	Có
P2	Classic	50	75	Không
P3	Motorcycle	30	40	Không
P4	Motorcycle	50	40	Có

Ghi nhớ

Cùng một biểu thức `AVG`, nhưng nhóm so sánh của P1 và P3 khác nhau.

Correlated COUNT: khách hàng có hơn 3 đơn

```sql
SELECT c.customerNumber, c.customerName
FROM customers AS c
WHERE (
    SELECT COUNT(*)
    FROM orders AS o
    WHERE o.customerNumber = c.customerNumber
) > 3;
```

Cách đọc

- Với từng khách hàng `c`, tính số đơn mang cùng `customerNumber`.
- Chỉ giữ khách hàng có `orderCount > 3`.

EXISTS: cú pháp và giải thích

```sql
SELECT o.column_list
FROM outer_table AS o
WHERE EXISTS (
    SELECT 1
    FROM inner_table AS i
    WHERE i.foreign_key = o.primary_key
);
```

- `EXISTS` cho `TRUE` khi subquery có ít nhất một dòng.
- `SELECT 1` là quy ước dễ đọc; nội dung biểu thức trong SELECT không quyết định điều kiện tồn tại.
- Phù hợp với câu hỏi *có ít nhất một …?*.

EXISTS: khách hàng đã từng đặt hàng

```sql
SELECT c.customerNumber, c.customerName
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customerNumber = c.customerNumber
);
```

Giải thích

- Một khách hàng được giữ khi có ít nhất một đơn hàng liên quan.
- Không cần đếm tổng số đơn nếu chỉ kiểm tra sự tồn tại.

EXISTS: sản phẩm có đơn hàng

```sql
SELECT p.productCode, p.productName
FROM products AS p
WHERE EXISTS (
    SELECT 1
    FROM orderdetails AS od
    WHERE od.productCode = p.productCode
);
```

Ghi nhớ

Kết quả giống ý định của `IN`, nhưng `EXISTS` biểu diễn trực tiếp câu hỏi có ít nhất một dòng chi tiết không.

EXISTS: mô phỏng từng dòng ngoài

Ví dụ giả lập:

Khách hàng	Số đơn tìm được	EXISTS?
C101	3	TRUE
C102	0	FALSE
C103	1	TRUE

EXISTS

Giữ `C101` và `C103`.

NOT EXISTS

Chỉ giữ `C102`.

NOT EXISTS: cú pháp

```sql
SELECT o.column_list
FROM outer_table AS o
WHERE NOT EXISTS (
    SELECT 1
    FROM inner_table AS i
    WHERE i.foreign_key = o.primary_key
);
```

- Trả về các dòng *không có* bản ghi phù hợp ở bảng liên quan.
- Khác với `NOT IN`, cách kiểm tra sự tồn tại không bị ảnh hưởng bởi NULL trong các giá trị không liên quan.

NOT EXISTS: khách hàng chưa có đơn hàng

```sql
SELECT c.customerNumber, c.customerName
FROM customers AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customerNumber = c.customerNumber
);
```

Ghi nhớ

Trong điều kiện so sánh, `o.customerNumber = c.customerNumber` là liên kết quan trọng nhất: bỏ nó sẽ kiểm tra xem *bất kỳ đơn hàng nào* tồn tại, không phải đơn của khách hiện tại.

NOT EXISTS: sản phẩm chưa từng được đặt

```sql
SELECT p.productCode, p.productName
FROM products AS p
WHERE NOT EXISTS (
    SELECT 1
    FROM orderdetails AS od
    WHERE od.productCode = p.productCode
);
```

- Phù hợp để phát hiện sản phẩm không xuất hiện trong chi tiết đơn.
- Kỹ thuật này còn gọi là kiểm tra *anti-existence*.

IN vs EXISTS vs NOT EXISTS: chọn kỹ thuật

Kỹ thuật	Câu hỏi điển hình	Lưu ý
`IN`	Khóa này có trong tập con?	Hợp với 1 cột nhiều dòng
`EXISTS`	Có ít nhất một dòng liên quan?	Thường là subquery tương quan
`NOT IN`	Khóa không thuộc tập con?	Cẩn thận NULL
`NOT EXISTS`	Không có dòng liên quan?	Ít rủi ro bẫy NULL

Ghi nhớ

Tính hiệu quả phụ thuộc dữ liệu, chỉ mục và phiên bản MySQL; không khẳng định phương pháp nào luôn nhanh nhất.

Derived table: cú pháp và vai trò

```sql
SELECT dt.column1, dt.aggregated_value
FROM (
    SELECT grouping_key AS column1,
           SUM(amount) AS aggregated_value
    FROM source_table
    GROUP BY grouping_key
) AS dt
WHERE dt.aggregated_value > threshold;
```

Giải thích

- Subquery trong `FROM` tạo một **quan hệ trung gian** nhiều dòng/cột.
- `AS dt` đặt bí danh; trong MySQL derived table cần có bí danh.
- Truy vấn ngoài tiếp tục lọc, sắp xếp hoặc nối dữ liệu của bảng trung gian.

Derived table: tổng thanh toán theo khách hàng

```sql
SELECT customerNumber, totalPayment
FROM (
    SELECT customerNumber,
           SUM(amount) AS totalPayment
    FROM payments
    GROUP BY customerNumber
) AS payment_summary
WHERE totalPayment > 100000;
```

1. Truy vấn trong gom `payments` theo `customerNumber`.
2. Tính `totalPayment` cho từng khách hàng.
3. Truy vấn ngoài giữ nhóm có tổng vượt 100000.

Derived table: diễn giải bảng trung gian

Bảng tạm giả lập do subquery tạo ra:

customerNumber	totalPayment
101	80,000
102	150,000
103	120,000
104	20,000

Subquery

Tính `SUM(amount)` theo mã khách.

Outer query

Giữ khách `102` và `103` (tổng > 100000).

Derived table: doanh thu từng đơn hàng

```sql
SELECT orderNumber, orderTotal
FROM (
    SELECT orderNumber,
           SUM(quantityOrdered * priceEach)
               AS orderTotal
    FROM orderdetails
    GROUP BY orderNumber
) AS order_summary
WHERE orderTotal > 50000;
```

Ghi nhớ

Doanh thu từng đơn dùng `SUM(quantityOrdered * priceEach)`; không nhầm với số tiền `payments.amount`.

Derived table và HAVING: so sánh

HAVING trực tiếp

```sql
SELECT customerNumber,
       SUM(amount) AS totalPayment
FROM payments
GROUP BY customerNumber
HAVING SUM(amount) > 100000;
```

Derived table

```sql
SELECT customerNumber, totalPayment
FROM (
  SELECT customerNumber,
         SUM(amount) AS totalPayment
  FROM payments
  GROUP BY customerNumber
) AS t
WHERE totalPayment > 100000;
```

Hai cách thể hiện cùng mục tiêu trên cùng nguồn dữ liệu, nhưng derived table thuận tiện nếu cần nối/biến đổi kết quả đã tổng hợp.

Bí danh derived table và phạm vi của cột

Thiếu alias:

```
SELECT *
FROM (
    SELECT customerNumber, SUM(amount) AS totalPayment
    FROM payments
    GROUP BY customerNumber
);
```

Sửa lại:

```
SELECT *
FROM (
    SELECT customerNumber, SUM(amount) AS totalPayment
    FROM payments
    GROUP BY customerNumber
) AS payment_summary;
```

Ghi nhớ

Derived table cần bí danh; chỉ những cột được `SELECT` bên trong mới dùng được ở ngoài.

Các lỗi subquery sinh viên thường mắc

Lỗi	Cách kiểm tra / sửa
`=` với nhiều dòng	Dùng `IN` hoặc giới hạn về một giá trị
`NOT IN` có NULL	Lọc NULL hoặc thay bằng `NOT EXISTS`
Thiếu liên kết correlated	Kiểm tra điều kiện bằng khóa hai bảng
Thiếu alias derived table	Thêm `AS tên_bảng_tạm`
`SUM()` kỳ vọng 0	Dùng `COALESCE(SUM(...),0)`
Lọc nhầm nhóm	Phân biệt `WHERE` và `HAVING`

Lỗi correlated: quên điều kiện liên kết

Truy vấn có lỗi logic:

```
SELECT c.customerNumber, c.customerName
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
);
```

Lưu ý

Nếu bảng orders có bất kỳ dòng nào, EXISTS sẽ đúng với mọi khách hàng; không phải "khách hàng đã đặt hàng.

Sửa: thêm `WHERE o.customerNumber = c.customerNumber` **ở subquery**.

Tư duy hiệu năng: subquery và chỉ mục

```sql
EXPLAIN
SELECT c.customerNumber, c.customerName
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customerNumber = c.customerNumber
);
```

- Chỉ mục trên khóa liên kết (ví dụ `orders.customerNumber`) thường hữu ích.
- Dùng `EXPLAIN` để xem kế hoạch thực thi thực tế.
- MySQL có thể biến đổi `IN`/`EXISTS` thành chiến lược semi-join, tùy truy vấn và phiên bản.

Luyện tập 1: scalar subquery (Bài 5.1)

Yêu cầu: Sản phẩm có MSRP lớn hơn MSRP trung bình.

```sql
SELECT productCode, productName, MSRP
FROM products
WHERE MSRP > (
    SELECT AVG(MSRP)
    FROM products
);
```

Câu hỏi trên lớp

Nếu một sản phẩm có `MSRP` đúng bằng trung bình, sản phẩm đó có được giữ lại không?

Luyện tập 2: IN (Bài 6.1)

Yêu cầu: Liệt kê khách hàng đã từng có đơn hàng.

```sql
SELECT customerNumber, customerName
FROM customers
WHERE customerNumber IN (
    SELECT customerNumber
    FROM orders
);
```

Ghi nhớ

Viết thêm một phiên bản sử dụng `EXISTS`, sau đó so sánh kết quả.

Luyện tập 3: NOT IN (Bài 7.4)

Yêu cầu: Nhân viên chưa phụ trách khách hàng nào.

```sql
SELECT employeeNumber, lastName, firstName
FROM employees
WHERE employeeNumber NOT IN (
    SELECT salesRepEmployeeNumber
    FROM customers
    WHERE salesRepEmployeeNumber IS NOT NULL
);
```

Ghi nhớ

Câu hỏi: vì sao không nên bỏ `IS NOT NULL`?

Luyện tập 4: subquery trong SELECT (Bài 8.1)

Yêu cầu: Mỗi khách hàng và số đơn hàng của khách đó.

```sql
SELECT c.customerNumber, c.customerName,
       (
           SELECT COUNT(*)
           FROM orders AS o
           WHERE o.customerNumber = c.customerNumber
       ) AS orderCount
FROM customers AS c;
```

Ghi nhớ

Khách hàng chưa có đơn sẽ có `orderCount = 0`.

Luyện tập 5: correlated AVG (Bài 9.3)

Yêu cầu: Giá mua cao hơn trung bình của cùng dòng sản phẩm.

```
SELECT p.productCode, p.productName,
       p.productLine, p.buyPrice
FROM products AS p
WHERE p.buyPrice > (
    SELECT AVG(p2.buyPrice)
    FROM products AS p2
    WHERE p2.productLine = p.productLine
);
```

Ghi nhớ

Đánh dấu hai bí danh `p` và `p2` thuộc phạm vi nào.

Luyện tập 6: EXISTS (Bài 10.1)

Yêu cầu: Khách hàng có ít nhất một đơn hàng.

```sql
SELECT c.customerNumber, c.customerName
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customerNumber = c.customerNumber
);
```

Ghi nhớ

Phần SELECT bên trong có thể là `SELECT 1`; quan trọng là subquery có dòng nào hay không.

Luyện tập 7: NOT EXISTS (Bài 11.3)

Yêu cầu: Sản phẩm chưa bao giờ được đặt.

```sql
SELECT p.productCode, p.productName
FROM products AS p
WHERE NOT EXISTS (
    SELECT 1
    FROM orderdetails AS od
    WHERE od.productCode = p.productCode
);
```

Ghi nhớ

Đây là cách an toàn và trực quan để mô tả “không có dòng liên quan.

Luyện tập 8: derived table (Bài 12.1)

Yêu cầu: Tổng thanh toán theo khách, chỉ giữ tổng > 100000.

```sql
SELECT customerNumber, totalPayment
FROM (
    SELECT customerNumber, SUM(amount) AS totalPayment
    FROM payments
    GROUP BY customerNumber
) AS payment_summary
WHERE totalPayment > 100000;
```

Ghi nhớ

Nêu tên bảng trung gian và hai cột mà nó cung cấp cho truy vấn ngoài.

Bài tập tự luyện sau giờ học

1. Dùng `NOT EXISTS` tìm khách hàng chưa từng thanh toán.
2. Dùng subquery tương quan tìm sản phẩm có tồn kho vượt trung bình cùng `productLine`.
3. Dùng derived table đếm số đơn theo khách hàng, lọc những khách có hơn 5 đơn.
4. Viết hai cách tương đương cho câu hỏi: dòng sản phẩm có ít nhất một sản phẩm (`IN` và `EXISTS`).
5. Tạo một ví dụ mà `NOT IN` cho kết quả ngoài dự kiến do `NULL`.

Ghi nhớ

Có thể dùng lab HTML SELECT 5 để thực hành Run/Check trên database mẫu.

Tổng kết: bản đồ lựa chọn subquery

Nhu cầu	Kỹ thuật gợi ý
So sánh với giá trị tổng hợp	Scalar subquery
Kiểm tra thuộc một tập giá trị	`IN`
Kiểm tra có ít nhất một dòng liên quan	`EXISTS`
Kiểm tra không có dòng liên quan	`NOT EXISTS`
Tính thêm cột cho từng dòng ngoài	Scalar correlated subquery trong `SELECT`
Tổng hợp trước, lọc/nối sau	Derived table trong `FROM`

Lưu ý

Đặc biệt chú ý `NOT IN` với `NULL`; kiểm tra số dòng trả về của scalar subquery và điều kiện liên kết khi correlated.

Tài liệu tham khảo và thực hành

- **Bài gốc:** *Lab: Subquery trong MySQL với classicmodels*.
  `https://vudmvn.github.io/dbms-course/lectures/MySQL/querying-data/select/select-statement-5/`
- Trang bài học có các nhóm bài thực hành **3.1–14.5** và các lời giải gợi ý.
- Lab HTML SELECT 5: chạy câu SQL, kiểm tra đáp án và theo dõi tiến độ.

Gợi ý giảng dạy

Dạy mỗi kỹ thuật qua chu trình: **vấn đề cần giải** → **cú pháp** → **cách đọc** → **Run thử** → **bài tập**.